

Java programming pattern programs

Developing understanding of **nested loops, rows/columns, conditions, and logic building.**

1. Solid Square Pattern

Output

```
* * * *  
* * * *  
* * * *  
* * * *
```

Java Code

```
public class SolidSquare {  
    public static void main(String[] args) {  
  
        int n = 4;  
  
        for (int i = 1; i <= n; i++) {  
            for (int j = 1; j <= n; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

2. Right-Angled Triangle

Output

```
*
* *
* * *
* * * *
* * * * *
```

Java Code

```
public class RightTriangle {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

Important logic:

Number of stars = current row number.

3. Inverted Right Triangle

Output

```
* * * * *
* * * *
* * *
* *
*
```

Java Code

```
public class InvertedTriangle {
    public static void main(String[] args) {

        int n = 5;

        for (int i = n; i >= 1; i--) {
            for (int j = 1; j <= i; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

4. Number Triangle

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Java Code

```
public class NumberTriangle {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
            }

            System.out.println();
        }
    }
}
```

5. Same Number Triangle

Output

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Java Code

```
public class SameNumberTriangle {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= i; j++) {
                System.out.print(i + " ");
            }

            System.out.println();
        }
    }
}
```

6. Floyd's Triangle

Output

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Java Code

```
public class FloydTriangle {
    public static void main(String[] args) {

        int n = 5;
        int number = 1;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= i; j++) {
                System.out.print(number + " ");
                number++;
            }

            System.out.println();
        }
    }
}
```

7. Right-Aligned Triangle

Output

```
      *
     * *
    * * *
   * * * *
  * * * * *
```

Java Code

```
public class RightAlignedTriangle {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            // Spaces
            for (int j = 1; j <= n - i; j++) {
                System.out.print("  ");
            }

            // Stars
            for (int j = 1; j <= i; j++) {
                System.out.print("* ");
            }

            System.out.println();
        }
    }
}
```

This is an important pattern because students learn that **one row can contain two separate inner loops**.

8. Pyramid Pattern

Output

```
  *
 ***
*****
*****
*****
```

Java Code

```
public class Pyramid {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            // Spaces
            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }

            // Stars
            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }

            System.out.println();
        }
    }
}
```

Important formula:

```
Spaces = n - i
Stars  = 2 * i - 1
```

9. Inverted Pyramid

Output

☆☆☆☆☆☆☆
 ☆☆☆☆☆
 ☆☆☆
 ☆☆☆
 ☆

Java Code

```
public class InvertedPyramid {
    public static void main(String[] args) {

        int n = 5;

        for (int i = n; i >= 1; i--) {

            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }

            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }

            System.out.println();
        }
    }
}
```

10. Diamond Pattern

Output

```
  *
 ***
*****
*****
*****
*****
***
 *
```

Java Code

```
public class Diamond {
    public static void main(String[] args) {

        int n = 5;

        // Upper half
        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }

            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }

            System.out.println();
        }

        // Lower half
        for (int i = n - 1; i >= 1; i--) {

            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }

            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }

            System.out.println();
        }
    }
}
```

11. Hollow Square

Output

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

Java Code

```
public class HollowSquare {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (i == 1 || i == n || j == 1 || j == n) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }
    }
}
```

This is an excellent pattern for introducing **if-else conditions inside nested loops**.

12. Hollow Triangle

Output

```
*
* *
*  *
*   *
* *  *
```

Java Code

```
public class HollowTriangle {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= i; j++) {

                if (j == 1 || j == i || i == n) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }

            System.out.println();
        }
    }
}
```

13. 0-1 Triangle

Output

```
1
01
101
0101
10101
```

Java Code

```
public class ZeroOneTriangle {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= i; j++) {

                if ((i + j) % 2 == 0) {
                    System.out.print("1");
                } else {
                    System.out.print("0");
                }
            }

            System.out.println();
        }
    }
}
```

14. Alphabet Triangle

Output

A
AB
ABC
ABCD
ABCDE

Java Code

```
public class AlphabetTriangle {  
    public static void main(String[] args) {  
  
        int n = 5;  
  
        for (int i = 1; i <= n; i++) {  
  
            for (int j = 1; j <= i; j++) {  
                System.out.print((char) ('A' + j - 1));  
            }  
  
            System.out.println();  
        }  
    }  
}
```

15. Repeated Alphabet Pattern

Output

```
A
BB
CCC
DDDD
EEEE
```

Java Code

```
public class RepeatedAlphabet {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= i; j++) {
                System.out.print((char) ('A' + i - 1));
            }

            System.out.println();
        }
    }
}
```

16. Palindrome Number Pyramid

Output

```
  1
 121
12321
1234321
123454321
```

Java Code

```
public class NumberPalindrome {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            // Spaces
            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }

            // Increasing numbers
            for (int j = 1; j <= i; j++) {
                System.out.print(j);
            }

            // Decreasing numbers
            for (int j = i - 1; j >= 1; j--) {
                System.out.print(j);
            }

            System.out.println();
        }
    }
}
```

Recommended Sequence

Level	Patterns	Main Concept
1	Solid Square	Nested <code>for</code> loops
2	Right Triangle	Row-dependent loop
3	Inverted Triangle	Decrementing loop
4	Number Triangle	Printing variables
5	Same Number Triangle	Outer-loop variable
6	Floyd's Triangle	Counter variable
7	Right-Aligned Triangle	Spaces + stars
8	Pyramid	Mathematical formula
9	Inverted Pyramid	Spaces + decreasing stars
10	Hollow Square	Conditions
11	Hollow Triangle	Conditions + loops
12	0-1 Triangle	Conditional logic
13	Alphabet Triangle	<code>char</code> and ASCII/Unicode values
14	Palindrome Pyramid	Multiple inner loops
15	Diamond	Combining patterns

The key concept to emphasize

Do not memorize pattern programs. Analyze every pattern using three questions:

1. **How many rows are there?**
2. **What should be printed in each row?**
3. **What is the relationship between the row number and the number of spaces/stars/numbers?**

For example, for a pyramid:

Row	Spaces	Stars
1	4	1
2	3	3
3	2	5
4	1	7
5	0	9

Therefore:

```
spaces = n - i;  
stars = 2 * i - 1;
```

This approach will help to **develop the logic themselves rather than simply copying pattern code.**